

03. Spring Boot Annotations

3.1. Giới thiệu Annotations trong Spring Boot

Annotations trong Spring Boot (và Spring Framework nói chung) là các siêu dữ liệu (metadata) được sử dụng để cung cấp thông tin cho trình biên dịch hoặc JVM, hoặc để cấu hình các thành phần của ứng dụng. Chúng giúp giảm thiểu việc cấu hình bằng XML hoặc Java code truyền thống, làm cho mã nguồn trở nên gọn gàng, dễ đọc và dễ bảo trì hơn. Spring Boot tận dụng mạnh mẽ các annotations để đơn giản hóa quá trình phát triển, đặc biệt là thông qua cơ chế tự động cấu hình.

Các annotations có thể được áp dụng cho các lớp, phương thức, trường, hoặc tham số. Chúng được xử lý trong thời gian biên dịch (compile-time) hoặc thời gian chạy (runtime) bởi Spring Framework để thực hiện các tác vụ như quản lý dependency injection, định nghĩa bean, xử lý web request, quản lý giao dịch, v.v.

3.2. Các Annotations cơ bản và phổ biến

3.2.1. @SpringBootApplication

Đây là annotation quan trọng nhất trong một ứng dụng Spring Boot. Nó là một annotation tiện lợi, kết hợp ba annotation khác:

- `@Configuration` : Đánh dấu lớp là một lớp cấu hình Spring, nơi bạn có thể định nghĩa các Spring beans.
- `@EnableAutoConfiguration` : Kích hoạt cơ chế tự động cấu hình của Spring Boot. Spring Boot sẽ tự động cấu hình các bean dựa trên các dependency có trong classpath và các thiết lập khác.
- `@ComponentScan` : Chỉ định Spring quét các component (như `@Component` , `@Service` , `@Repository` , `@Controller` , `@RestController`) trong package hiện tại và các package con để tìm và đăng ký các bean.

Ví dụ:

Java

```
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class MyApplication {
    public static void main(String[] args) {
        SpringApplication.run(MyApplication.class, args);
    }
}
```

3.2.2. Annotations về Component Scan

- `@Component` : Annotation chung để đánh dấu một lớp là một Spring component. Các lớp được đánh dấu bằng `@Component` sẽ được Spring container quản lý và tạo bean.
- `@Service` : Là một chuyên biệt hóa của `@Component` , dùng để đánh dấu các lớp trong tầng dịch vụ (service layer). Nó không có thêm chức năng kỹ thuật nào so với `@Component` nhưng giúp phân biệt rõ ràng vai trò của lớp.
- `@Repository` : Là một chuyên biệt hóa của `@Component` , dùng để đánh dấu các lớp trong tầng truy cập dữ liệu (data access layer). Nó cũng cung cấp tính năng dịch các ngoại lệ cụ thể của cơ sở dữ liệu thành các ngoại lệ `DataAccessException` của Spring.
- `@Controller` : Là một chuyên biệt hóa của `@Component` , dùng để đánh dấu các lớp trong tầng điều khiển (controller layer) trong ứng dụng Spring MVC. Các lớp này xử lý các request web và trả về view.
- `@RestController` : Là một chuyên biệt hóa của `@Controller` và `@ResponseBody` . Được sử dụng để xây dựng các RESTful web services. Các phương thức trong lớp này sẽ trả về dữ liệu trực tiếp (ví dụ: JSON, XML) thay vì tên view.

Ví dụ:

Java

```
@RestController
@RequestMapping("/api/products")
public class ProductController {
    // ...
}
```

```

}

@Service
public class ProductService {
    // ...
}

@Repository
public class ProductRepository {
    // ...
}

```

3.2.3. Annotations về Dependency Injection

- `@Autowired` : Được sử dụng để tự động tiêm (inject) các dependency vào một bean. Spring sẽ tìm kiếm một bean phù hợp với kiểu dữ liệu của trường, constructor hoặc setter method và tiêm nó vào.

Ví dụ:

Java

```

@Service
public class OrderService {

    private final ProductRepository productRepository;

    @Autowired // Có thể bỏ qua nếu chỉ có một constructor
    public OrderService(ProductRepository productRepository) {
        this.productRepository = productRepository;
    }

    // Hoặc trên trường (ít được khuyến khích hơn)
    // @Autowired
    // private ProductRepository productRepository;
}

```

- `@Qualifier` : Được sử dụng kết hợp với `@Autowired` khi có nhiều hơn một bean cùng kiểu dữ liệu. Nó giúp chỉ định rõ bean nào cần được tiêm.

Ví dụ:

Java

```

@Component("smsSender")
public class SmsNotificationSender implements NotificationSender { /* ... */
}

@Component("emailSender")
public class EmailNotificationSender implements NotificationSender { /* ...
*/ }

@Service
public class NotificationService {

    @Autowired
    @Qualifier("emailSender")
    private NotificationSender sender;

    // ...
}

```

3.2.4. Annotations về Web (Spring MVC/WebFlux)

- `@RequestMapping` : Ánh xạ các request HTTP đến các phương thức xử lý trong controller. Có thể được sử dụng ở cấp độ lớp (để định nghĩa base path) hoặc cấp độ phương thức.
- `@GetMapping` , `@PostMapping` , `@PutMapping` , `@DeleteMapping` , `@PatchMapping` : Các annotation chuyên biệt của `@RequestMapping` cho các phương thức HTTP tương ứng (GET, POST, PUT, DELETE, PATCH). Giúp mã nguồn rõ ràng hơn.

Ví dụ:

Java

```

@RestController
@RequestMapping("/api/users")
public class UserController {

    @GetMapping // GET /api/users
    public List<User> getAllUsers() { /* ... */ }

    @GetMapping("/{id}") // GET /api/users/{id}
    public User getUserById(@PathVariable Long id) { /* ... */ }

    @PostMapping // POST /api/users
    public User createUser(@RequestBody User user) { /* ... */ }
}

```

```

    @PutMapping("/{id}") // PUT /api/users/{id}
    public User updateUser(@PathVariable Long id, @RequestBody User user) {
        /* ... */
    }

    @DeleteMapping("/{id}") // DELETE /api/users/{id}
    public void deleteUser(@PathVariable Long id) { /* ... */ }
}

```

- `@PathVariable` : Trích xuất giá trị từ phần đường dẫn của URL.
- `@RequestParam` : Trích xuất giá trị từ các tham số truy vấn (query parameters) trong URL.
- `@RequestBody` : Ánh xạ nội dung của request body (thường là JSON hoặc XML) vào một đối tượng Java.
- `@ResponseBody` : Cho biết giá trị trả về của phương thức nên được ánh xạ trực tiếp vào response body của HTTP (thường là JSON hoặc XML). `@RestController` đã bao gồm `@ResponseBody` .
- `@ResponseStatus` : Đặt mã trạng thái HTTP cho phản hồi. Thường được sử dụng trên các phương thức xử lý ngoại lệ hoặc khi trả về một trạng thái cụ thể.

Ví dụ:

Java

```

@GetMapping("/{id}")
public ResponseEntity<User> getUserById(@PathVariable Long id) {
    User user = userService.findById(id);
    if (user == null) {
        throw new UserNotFoundException("User not found with ID: " + id);
    }
    return ResponseEntity.ok(user);
}

@ResponseStatus(HttpStatus.NOT_FOUND)
public class UserNotFoundException extends RuntimeException {
    public UserNotFoundException(String message) {
        super(message);
    }
}

```

3.2.5. Annotations về Cấu hình

- `@Configuration` : Đánh dấu một lớp là một nguồn định nghĩa bean. Các phương thức được đánh dấu `@Bean` trong lớp này sẽ tạo ra các Spring beans.
- `@Bean` : Đánh dấu một phương thức trong lớp `@Configuration` sẽ tạo ra và trả về một Spring bean.

Ví dụ:

Java

```
@Configuration
public class AppConfig {

    @Bean
    public MyService myService() {
        return new MyServiceImpl();
    }

    @Bean
    public DataSource dataSource() {
        // Cấu hình DataSource
        return new HikariDataSource();
    }
}
```

- `@Value` : Tiêm giá trị từ các thuộc tính cấu hình (ví dụ: từ `application.properties` hoặc biến môi trường) vào một trường hoặc tham số.

Ví dụ:

Java

```
@Component
public class AppInfo {
    @Value("${app.name}")
    private String appName;

    @Value("${app.version:1.0.0}") // Giá trị mặc định nếu không tìm thấy
    private String appVersion;
}
```

```
// ...  
}
```

- `@ConfigurationProperties` : Ánh xạ một nhóm các thuộc tính cấu hình có tiền tố chung vào một lớp Java POJO. Rất hữu ích để quản lý các cấu hình phức tạp.

Ví dụ:

Java

```
@ConfigurationProperties(prefix = "app.security")  
@Component  
public class SecurityProperties {  
    private String secretKey;  
    private List<String> allowedOrigins;  
  
    // Getters and Setters  
}
```

3.2.6. Annotations về Data (JPA/Hibernate)

- `@Entity` : Đánh dấu một lớp là một entity JPA, ánh xạ tới một bảng trong cơ sở dữ liệu.
- `@Table` : Chỉ định tên bảng trong cơ sở dữ liệu nếu nó khác với tên lớp.
- `@Id` : Đánh dấu trường là khóa chính của entity.
- `@GeneratedValue` : Cấu hình cách giá trị khóa chính được tạo ra (ví dụ: IDENTITY, AUTO, SEQUENCE).
- `@Column` : Chỉ định tên cột, độ dài, có cho phép null hay không, v.v.
- `@OneToOne` , `@OneToMany` , `@ManyToOne` , `@ManyToMany` : Định nghĩa các mối quan hệ giữa các entity.
- `@Transactional` : Đánh dấu một phương thức hoặc lớp là một phần của giao dịch (transaction). Spring sẽ tự động quản lý việc bắt đầu, commit hoặc rollback giao dịch.

Ví dụ:

Java

```

@Entity
@Table(name = "products")
public class Product {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "product_name", nullable = false, length = 100)
    private String name;

    private double price;

    // Getters and Setters
}

@Repository
public interface ProductRepository extends JpaRepository<Product, Long> {
    // ...
}

@Service
public class ProductService {
    @Autowired
    private ProductRepository productRepository;

    @Transactional
    public Product saveProduct(Product product) {
        return productRepository.save(product);
    }

    @Transactional(readOnly = true)
    public List<Product> getAllProducts() {
        return productRepository.findAll();
    }
}

```

3.3. Bản chất của Annotations trong Spring Boot

Bản chất của annotations trong Spring Boot là cung cấp một cách khai báo (declarative) để cấu hình và định nghĩa các thành phần của ứng dụng. Thay vì sử dụng các file XML dài dòng hoặc Java code cấu hình phức tạp, annotations cho phép bạn nhúng trực tiếp các thông tin cấu hình vào mã nguồn, làm cho mã nguồn trở nên tự mô tả và dễ hiểu hơn. Điều này không chỉ giúp giảm thiểu boilerplate code mà còn tăng tốc độ phát triển, đặc biệt khi kết hợp với

cơ chế tự động cấu hình của Spring Boot. Annotations là một phần không thể thiếu trong việc xây dựng các ứng dụng Spring Boot hiện đại, thúc đẩy một phong cách lập trình rõ ràng, ngắn gọn và hiệu quả.